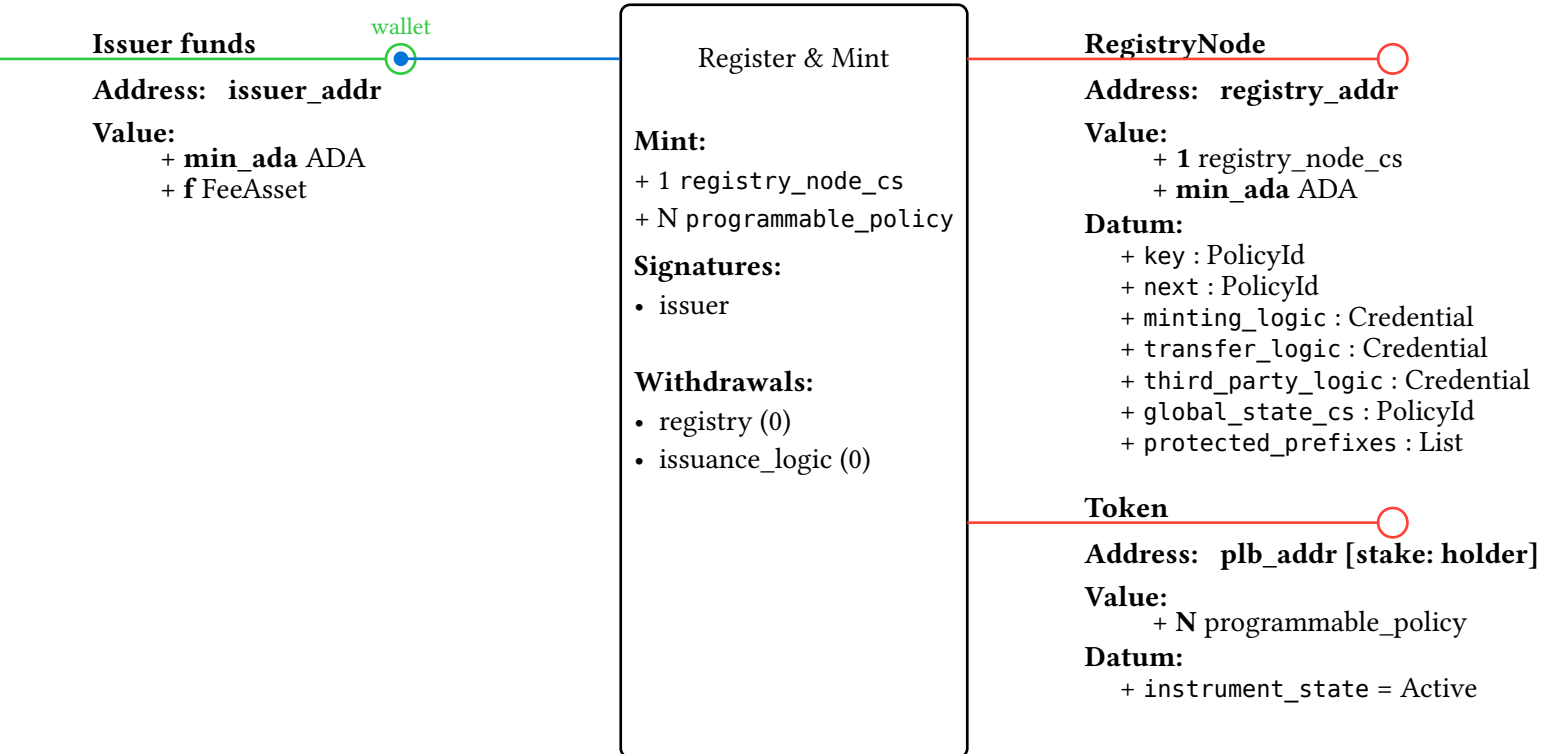


Contents

1 - Register & Mint (P1 base + P2 hooks)	2
2 - Permissive transfer (P1)	3
3 - Event rule fires (P2)	4
4 - Graduation / unwrap (P3)	5

1 - Register & Mint (P1 base + P2 hooks)

Registers the CIP-113 policy, plants a permissive transfer_logic (P1) and an event-rule Credential (P2) in the token's RegistryNode, and optionally mints the first batch in the same transaction.



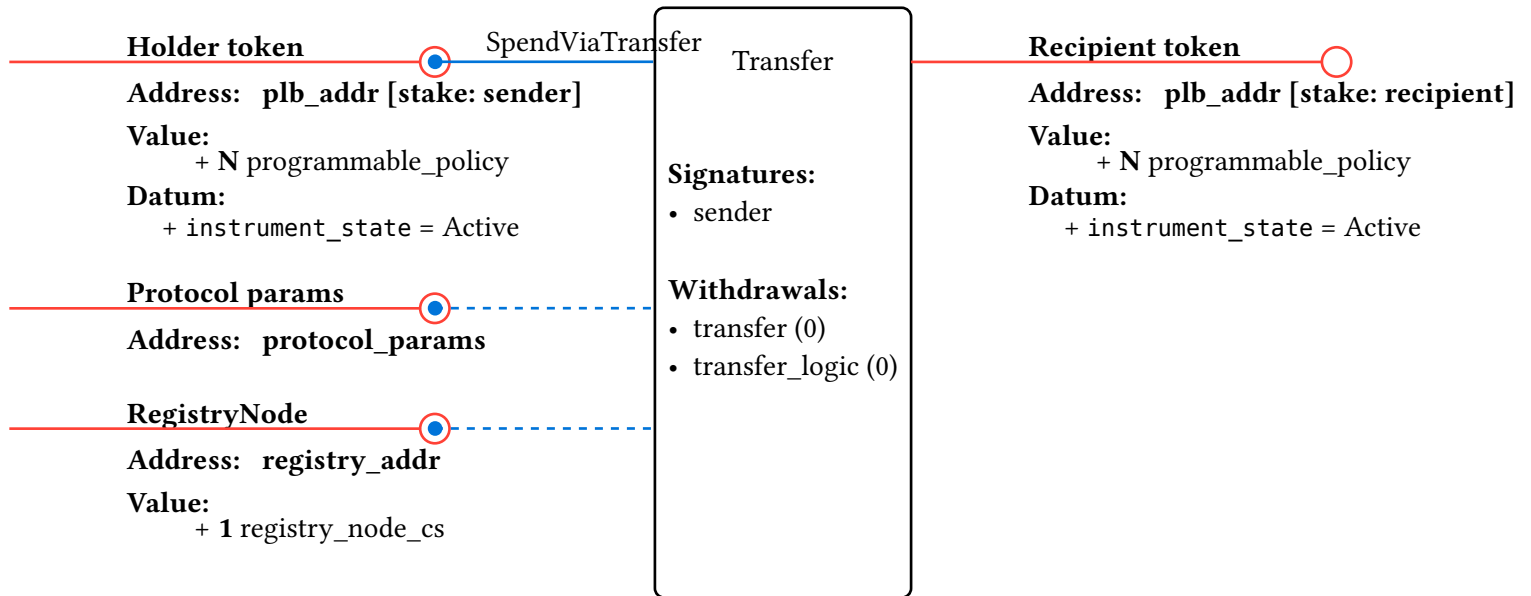
Notes:

- registry_mint binds minting_logic_script to the issuance_mint policy (expect_programmable_token_id_valid); it is frozen for the node's life.
- RegistryNode fields are drawn short: minting_logic / transfer_logic / third_party_logic are the canonical *_logic_script Credentials. protected_prefixes is the CIP-67 label list (100 / 500) the third-party path may not seize.
- transfer_logic is the permissive P1 predicate; the attached event rule (P2) is a pluggable Credential (issue 11, ARCHITECTURE.md §3). third_party_logic is the admin/seizure path, not the P2 event hook.
- Register-only and atomic register + mint are both allowed. A mode-aware issuance validator (Register / UpdateNode / Mint / Burn) decides which is permitted (Q-GRAD-2).
- Tokens are minted to the PLB address with the recipient's stake credential ([stake: holder]); the global validator forbids moving them to non-programmable addresses.

Figure 1: Register and mint transaction

2 - Permissive transfer (P1)

The token moves as freely as a native asset. The transfer core validator checks ownership, value preservation and the registry, then dispatches to the substandard's `transfer_logic` predicate, which approves unconditionally.



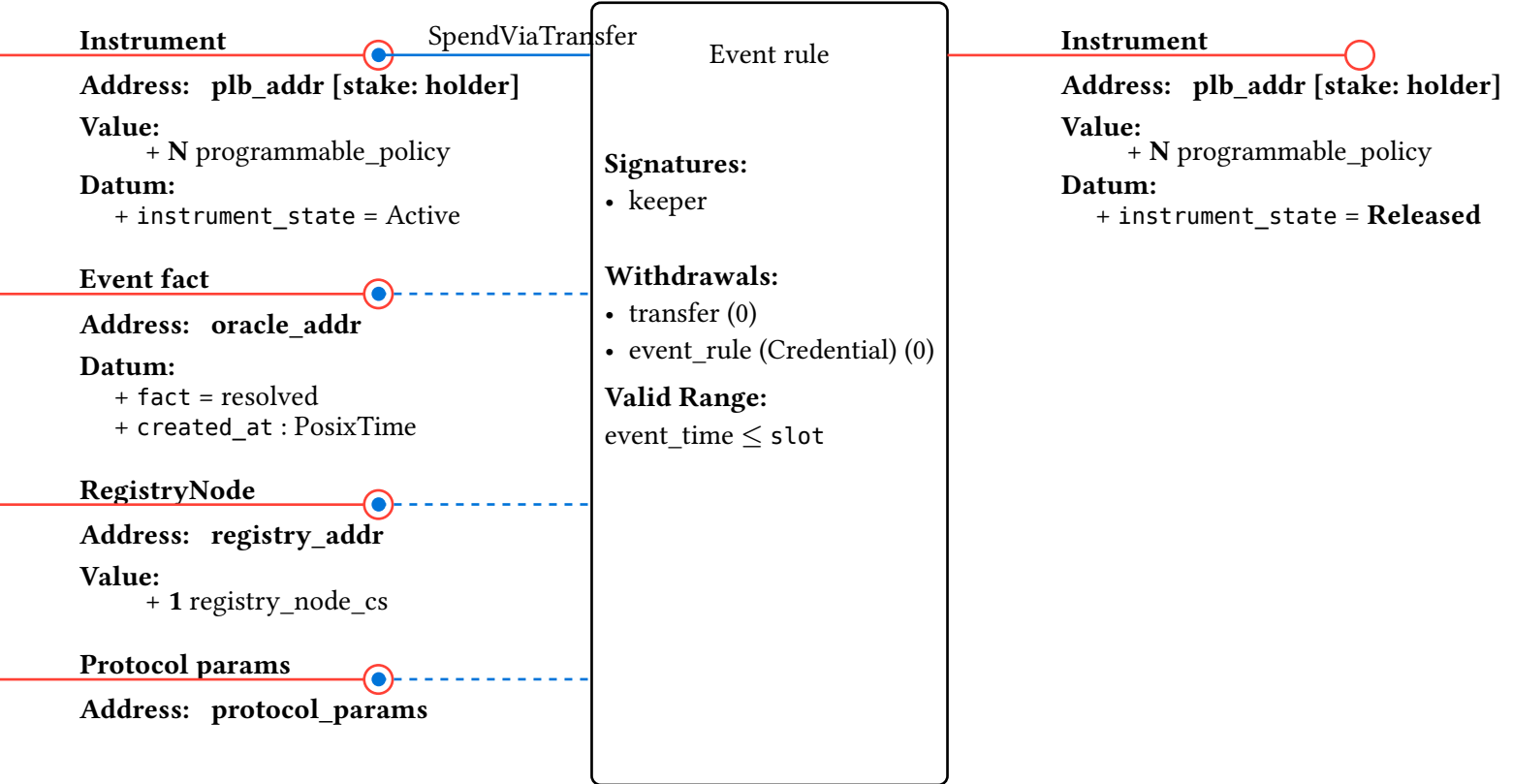
Notes:

- The PLB address carries the holder's stake credential: `transfer` verifies the sender signed and the continuation output carries the recipient, so ownership moves with the token.
- No transfer gate: `transfer_logic` is permissive, so programmability never restricts who may hold or send the token (P1).
- The core transfer stake validator performs ownership verification, value preservation and registry lookup; the substandard only supplies custom rules.
- P2 event predicates read their event state as reference inputs and do not gate an ordinary transfer.

Figure 2: Permissive transfer transaction

3 - Event rule fires (P2)

On a discrete event (time cliff, market resolution, maturity, attestation) a keeper submits a transaction that references the event state. The P2 predicate approves the resulting state change if and only if the event condition holds. Nothing self-executes.



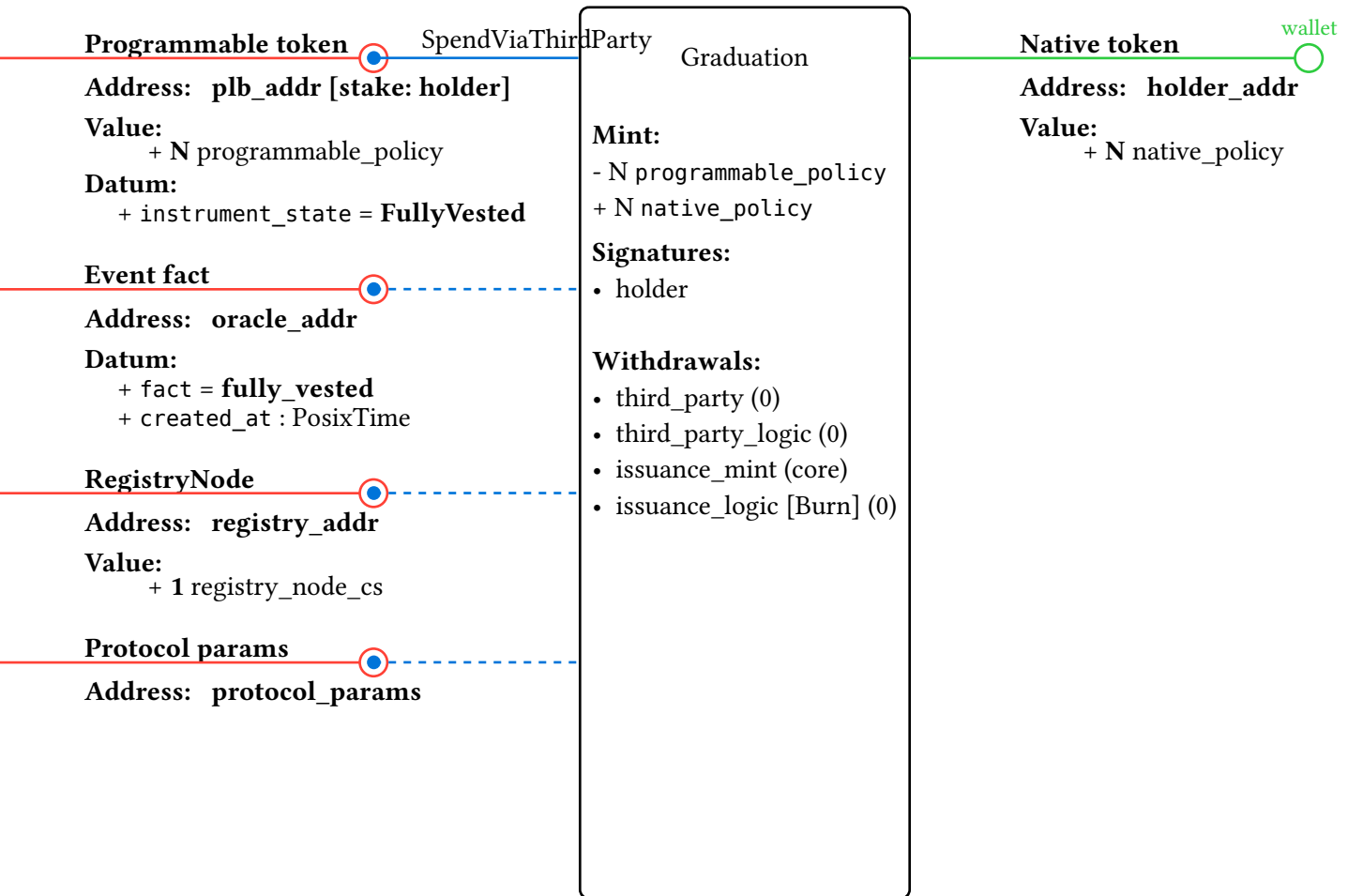
Notes:

- Reactive, not autonomous: a keeper detects the event and submits; the predicate only approves or rejects. There is no scheduler.
- The event hook is a pluggable logic (Credential) dispatched by the transfer path, not the third-party/admin path: a key is satisfied by a signature, a script by a forwarded withdraw-0 (issue 11).
- Oracles are pull-based; the rule reads the fact as a reference input and must itself check freshness against `created_at`.
- A validity range enforces time-based conditions. Per-holder state is contention-free; shared-state UTxOs serialize (Q-CONTENTION-1).
- The continuation stays under the same holder stake credential; only the instrument state changes.

Figure 3: Event-rule transaction

4 - Graduation / unwrap (P3)

End of life. An event-gated transaction, executed through the third-party path, force-burns the CIP-113 token (authorized by the substandard's issuance logic) and mints the equivalent plain native token under a separate policy, removing the contract dependency.



Notes:

- Graduation is executed through the third-party path (SpendViaThirdParty) to force-burn holders' tokens. All programmable tokens are custodied at the PLB, so it must burn the CIP-113 token and mint a distinct native token.
- The holder must sign (Q-GRAD-2): the forced action is authorized only with the holder's consent, so graduation is settlement, not a discretionary seizure.
- Authorized by the substandard's issuance logic (minting_logic_script), frozen at registration, so the gating must be designed into the mode-aware validator up front (Q-GRAD-2). Event-gated by the same condition the P2 rules enforce.
- Base-layer guarantee: a transaction that spends a registry node can never mint or burn that node's own token, so graduation is always pure issuance, never mixed with a registry reconfiguration.
- Burn + mint is the established migration shape; Q-GRAD-1 tracks asset identity and fungibility across the flip.

Figure 4: Graduation transaction